

CS3.301: Operating Systems and Networks

Lecture 7: CPU Scheduling (Linux CF Scheduler and Multiprocessor Scheduling)

INSTRUCTOR: DR. DEEPAK GANGADHARAN

Proportional Share scheduler

- Also sometimes referred to as **fair-share scheduler**
- **Concept: Scheduler tries to guarantee that each job gets a certain percentage of CPU time**
- Examples: Lottery Scheduler, Stride Scheduler, **Linux Completely Fair scheduler**
- **CFS (Completely Fair Scheduler): Linux's answer — fair-share scheduling built for efficiency and scale**
- Goal: spend as little time as possible making scheduling decisions
- Motivation: Google datacenter study found scheduling alone consumes ~5% of all datacenter CPU time

Linux CF Scheduler

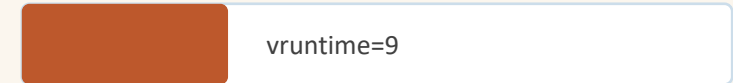
- CFS's goal: fairly divide the CPU evenly among all competing processes
- Achieves through a simple counting-based technique known as **virtual runtime (vruntime)**
- **Each process accumulates vruntime as it runs**
- In the basic case, vruntime grows at the same rate as physical time for every process
- **At every scheduling decision, CFS picks the process with the LOWEST vruntime to run next**
- No fixed time slice — CFS dynamically decides how long to run each process

Who Runs Next?

A



B



C



D



B has the lowest vruntime → B runs next

sched_latency: Setting the Time Slice

- Switch too often → great near-term fairness, but excessive context-switch overhead
- Switch too rarely → better performance, but poor near-term fairness
- CFS resolves this tension with a control parameter: sched_latency
- sched_latency is the target window over which CFS guarantees complete fairness
- Typical value: 48 ms

Formula

$$\text{time_slice} = \text{sched_latency} / n$$

where n = number of currently runnable processes

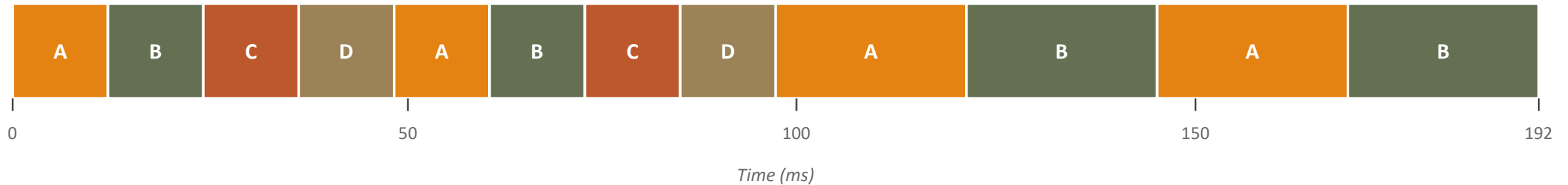
Worked Example: $n = 4$ processes

sched_latency	48 ms
n (processes)	4
time_slice per process	$48 / 4 = 12$ ms

Over 48 ms, each of the 4 processes runs exactly once — perfectly fair.

CFS in Action — Figure 9.4

Four jobs (A, B, C, D) share the CPU with sched_latency = 48 ms. Each runs for 12 ms before CFS re-checks vruntime.



- First 96 ms: A, B, C, D each get a 12 ms slice, round after round — every process's vruntime rises together
- **At t = 96 ms, jobs C and D complete**
- Only A and B remain — CFS now gives each of them the full remaining share: 24 ms per turn, round robin
- The mechanism never changed — CFS just kept picking the lowest vruntime job each time

Too Many Processes: min_granularity

- What if there are “too many” processes?
- Dividing sched_latency by a large n gives a tiny time slice → excessive context switching
- CFS adds a floor: min_granularity (typically 6 ms)
- CFS never sets a time slice below this value, no matter how many processes are runnable
- Trade-off: CFS won't be quite perfectly fair over the full sched_latency window, but stays close — while protecting CPU efficiency

Worked Example: n = 10 processes

Naive calculation

$$48 \text{ ms} / 10 = 4.8 \text{ ms}$$

min_granularity floor

6 ms

Actual time slice used

6 ms per process

Priority via Niceness

- CFS controls priority not with tickets, but the classic UNIX nice value
- Range: -20 (highest priority) to +19 (lowest priority); default is 0
- Every nice value maps to a fixed weight (a lookup table baked into the kernel)
- **Key property: the table preserves proportionality — a constant nice-value gap always produces the same CPU ratio**

Nice	-20	-10	-5	0	5	10	19
Weight	88761	9548	3121	1024	335	110	15

Excerpt from prio_to_weight[] — higher priority (lower nice) → much larger weight → more CPU share

Why nice = 0 → weight = 1024?

1024 is the default weight (`weight0`) — it's the baseline every other process's share is compared against. A process with double the weight of the default gets roughly double the CPU time.

Time Slice & vruntime with Weights

Time Slice Formula

$$\text{time_slice}_k = (\text{weight}_k / \sum \text{weight}_i) \times \text{sched_latency}$$

vruntime Update Formula

$$\text{vruntime}_i += (\text{weight}_0 / \text{weight}_i) \times \text{runtime}_i$$

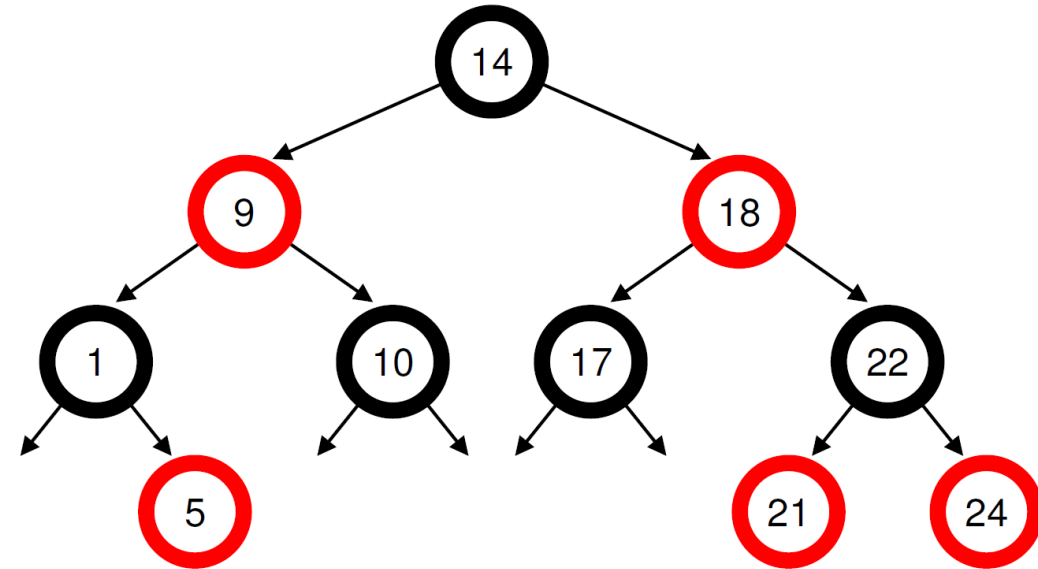
Worked Example — Job A (nice = -5) vs. Job B (nice = 0)

Job	Nice	Weight	Time Slice (of 48ms)	vruntime rate
A	-5	3121	≈ 36 ms (3/4)	1024/3121 ≈ 0.33×
B	0	1024	≈ 12 ms (1/4)	1024/1024 = 1.00×

A gets 3× the time slice of B, and A's vruntime accumulates 3× slower — both effects reinforce each other, giving A a consistently larger CPU share.

Keeping Processes in a Red-Black Tree

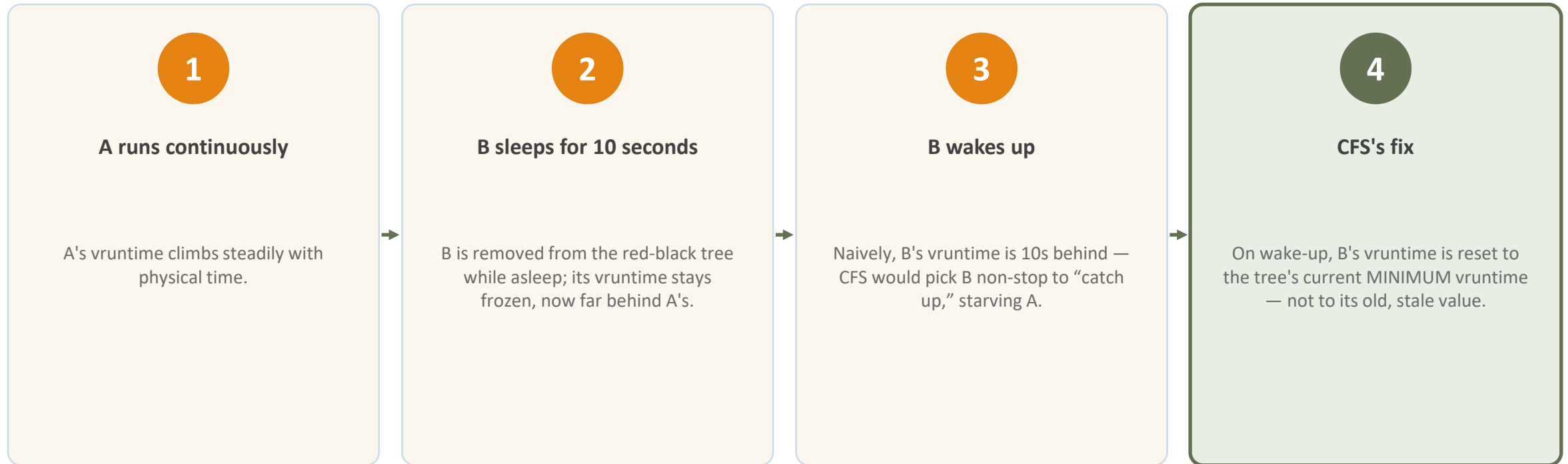
- Finding the next job (lowest vruntime) must be FAST — modern systems can have 1000s of processes
- **A simple sorted list needs $O(n)$ time to insert a process back in order**
- **CFS instead uses a red-black tree, ordered by vruntime**
- Insertion, deletion, and find-minimum are all $O(\log n)$
- Only running/runnable processes live in the tree — sleeping processes are tracked elsewhere



Leftmost node = lowest vruntime = next process to run

Sleeping Processes: Avoiding Starvation

Problem: a process that sleeps for a long time falls far behind on vruntime — and could monopolize the CPU when it wakes.



- **Trade-off: this avoids starving CPU-bound jobs, but jobs that sleep frequently for short periods still may not get their exact fair share**

Other CFS Features

Cache-Aware Heuristics

Numerous heuristics to improve cache performance when deciding which CPU / core runs which process.

Multi-CPU Support

Strategies for load-balancing and migration across multiple CPUs (building on multiprocessor scheduling).

Group Scheduling

Can schedule fairly across large groups of processes, not just individual processes.

Timer-Driven

Relies on a periodic timer interrupt (e.g., every 1 ms) to check if the running job's slice has expired.

Limitations, and What Comes Next

- Fair-share schedulers don't mesh perfectly with I/O
 - Jobs that perform I/O occasionally may still not get their exact fair share
- As of Linux 6.6, CFS was replaced as the default by EEVDF (Earliest Eligible Virtual Deadline First)
 - EEVDF is also a fair-sharing approach — the ticket/weight ideas here remain foundational

Linux Scheduler Timeline



CFS at a Glance

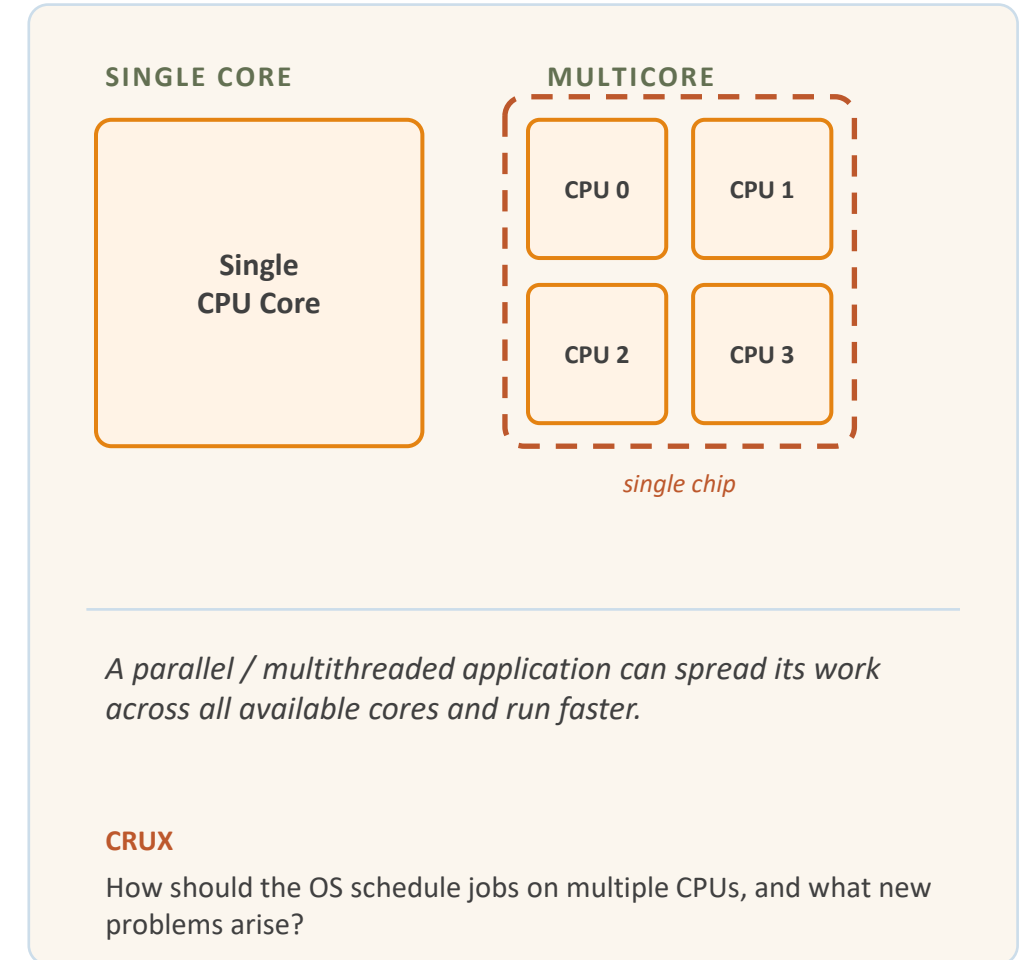
Mechanism	Purpose
vruntime	Tracks each process's accumulated (weighted) CPU time; lowest vruntime runs next
sched_latency	Target window (~48 ms) over which CFS aims to be completely fair
min_granularity	Floor on time slice (~6 ms) to bound context-switch overhead
nice / weight	Lets processes get a proportionally larger or smaller CPU share
Red-black tree	$O(\log n)$ insert / delete / find-min — scales to thousands of processes
Wake-up vruntime reset	Prevents long-sleeping processes from starving others when they wake

CFS: weighted round-robin with dynamic time slices, engineered to scale — the most widely used Linux fair-share scheduler for many years.

Multiprocessor Scheduling

Multicore Is Now the Default

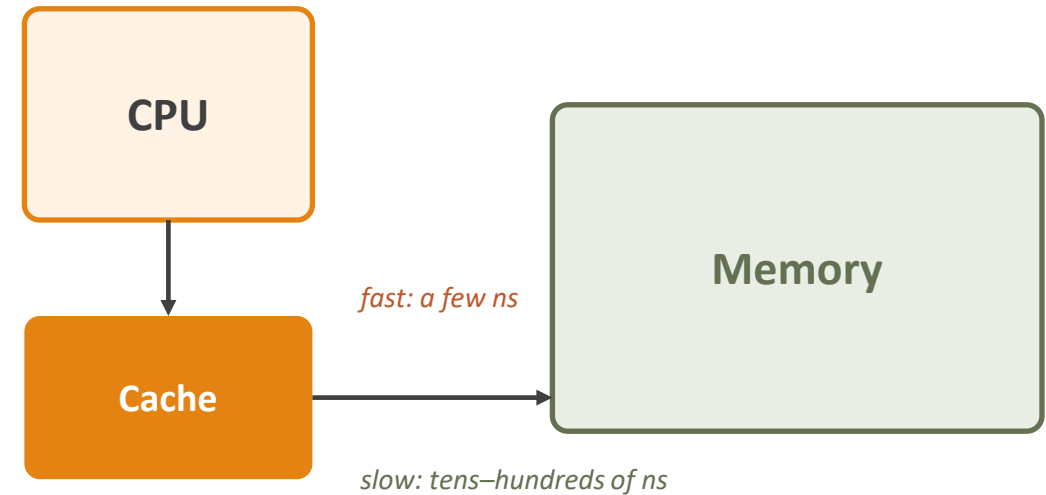
- Multiprocessor systems used to exist only at the high end of computing
- Now commonplace: desktops, laptops, and even mobile devices
- Architects struggled to make a single CPU much faster without using far too much power
- The fix: pack multiple CPU cores onto a single chip — the multicore processor
- **But a typical single-threaded program only ever uses one core — more cores alone don't make it run faster**
- Applications must be rewritten to run in parallel (e.g., using threads) to actually benefit



Single CPU with Cache

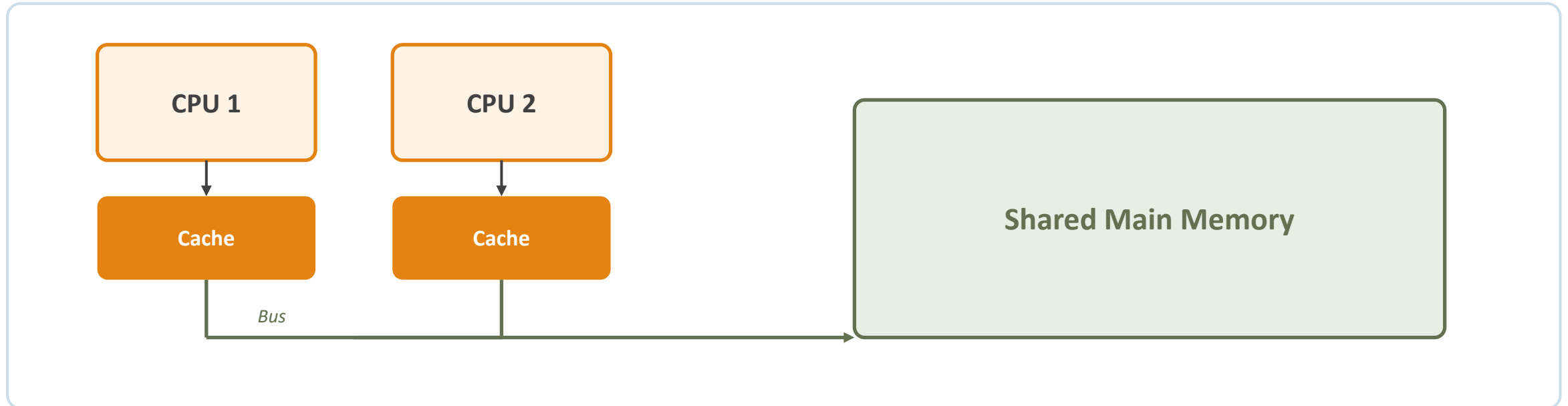
- Caches are small, fast memories holding copies of popular data from main memory
- Main memory is large but slow; caches make it appear fast
- Temporal locality — data accessed once is likely accessed again soon
- Spatial locality — data near address x is likely accessed soon too
- First load: fetched from memory (tens–hundreds of ns)
- Later loads: served from the cache (just a few ns)

Figure 10.1



A load instruction first checks the cache; on a miss it goes to memory, and the result is cached for next time.

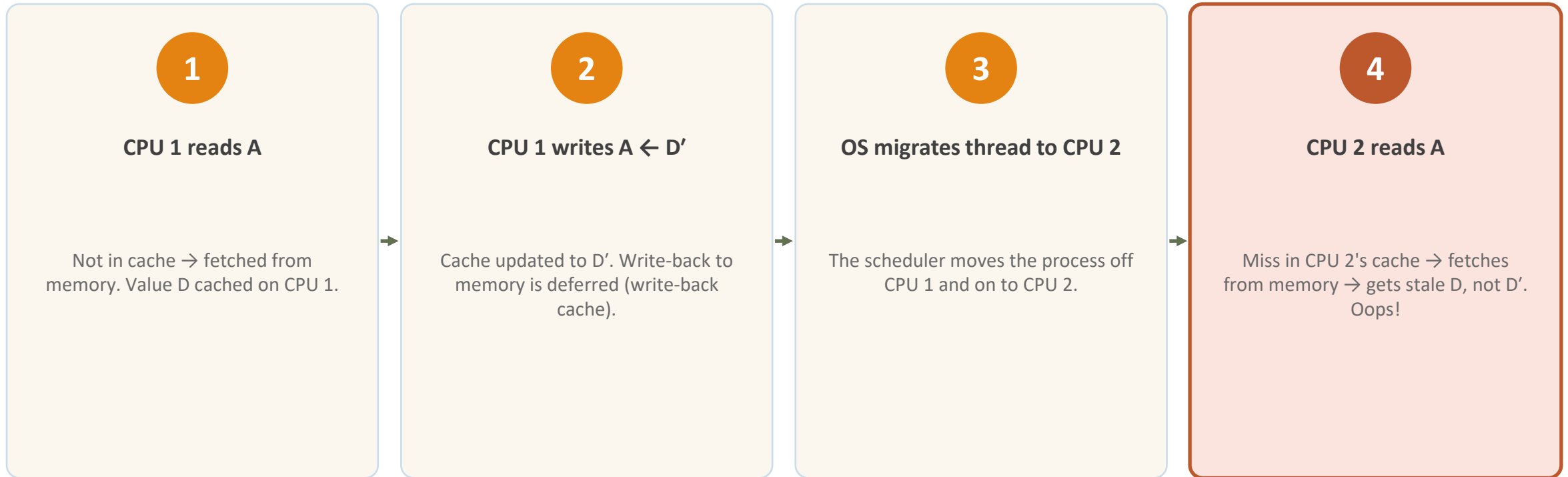
Multiple CPUs, Shared Memory — Figure 10.2



- **What happens when both CPUs cache the same data — and one of them writes a new value?**
- CPU 1 reads address A → value D is fetched from memory and cached
- CPU 1 writes A → its cache holds D', but the write-back to memory is delayed
- The OS moves the thread to CPU 2 → CPU 2 re-reads A, misses in its own cache, and fetches the stale value D from memory, not D'

The Cache Coherence Problem — Timeline

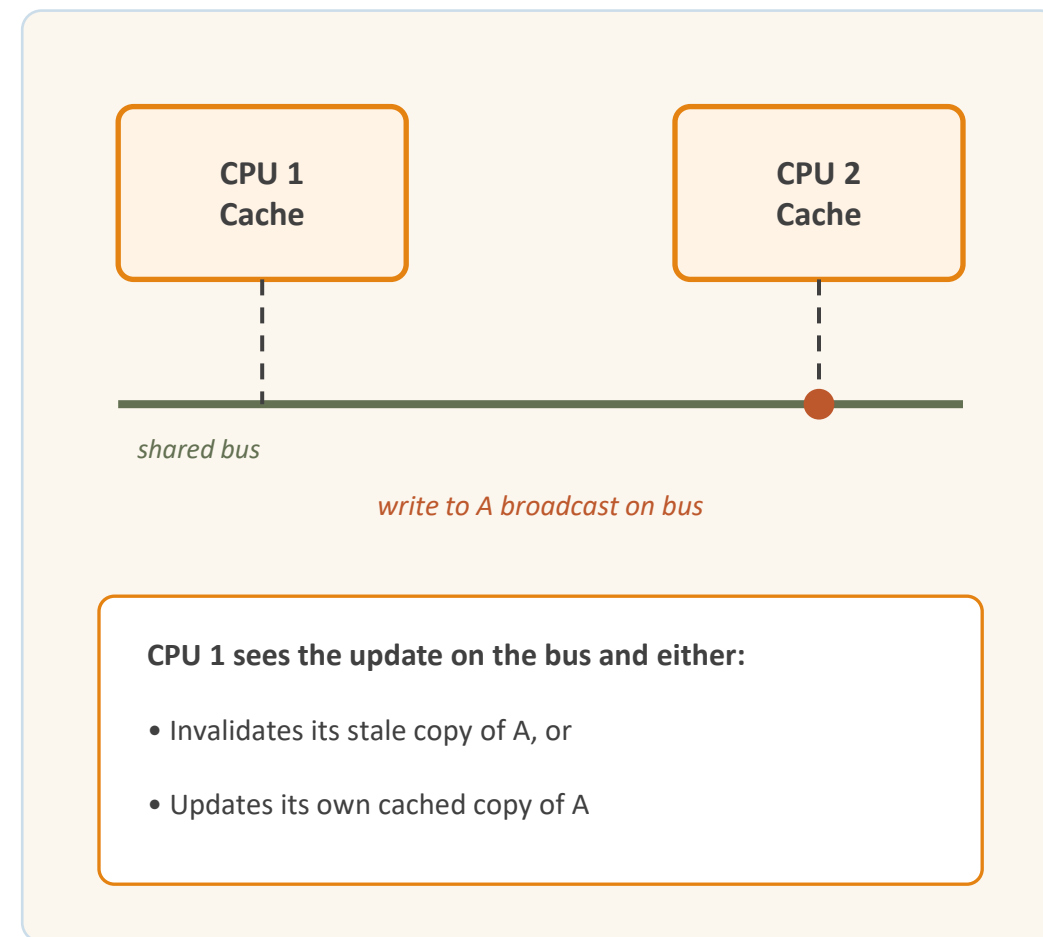
This is the cache-coherence problem: without help, per-CPU caches break the illusion of a single shared memory.



Result: CPU 2 silently computes with wrong data. Hardware must guarantee this cannot happen unnoticed.

Solving Coherence: Bus Snooping

- Hardware monitors memory accesses to preserve the view of a single shared memory
- **Bus snooping (Goodman, 1983): each cache watches the shared bus for updates**
- When a CPU sees an update to a value it holds, it reacts by either:
 - Invalidating — removing the stale copy from its own cache
 - Updating — installing the new value into its own cache
- Write-back caches complicate this, since the write to memory isn't visible until later
- A deep topic of its own — see a computer architecture course for full protocol families



Don't Forget Synchronization

Cache coherence keeps caches consistent — it does NOT make concurrent updates to shared structures correct. Locks are still required.

Figure 10.3 — Simple List Delete

```
int List_Pop() {  
    Node_t *tmp = head;    // (1)  
    int value = head->value; // (2)  
    head = head->next;     // (3)  
    free(tmp);             // (4)  
    return value;  
}
```

Without a lock: two threads race



Both threads capture the SAME node — both will free() it and may return the same value twice.

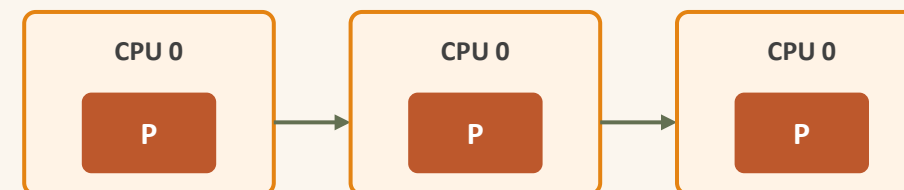
Fix: lock(&m) ... unlock(&m) around the routine

Lesson: as CPU count grows, contention on such a lock also grows — this will matter for scheduler design.

Cache Affinity

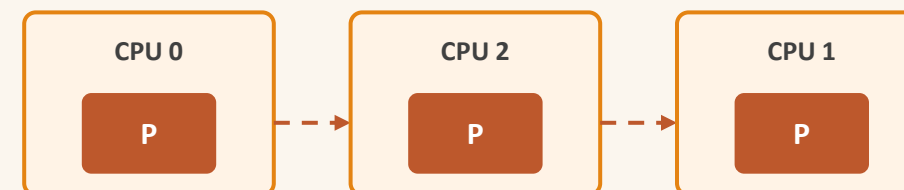
- A process running on a CPU builds up state in that CPU's caches and TLBs
- **Cache affinity: it usually runs faster if scheduled again on the same CPU**
- Run it on a different CPU each time, and it must reload that state — slower (though still correct, thanks to coherence)
- A good multiprocessor scheduler should consider affinity when making decisions
- This tension — keep a job put vs. balance load — will resurface throughout this lecture

Same CPU each time — warm cache



Cache stays warm → process keeps running fast

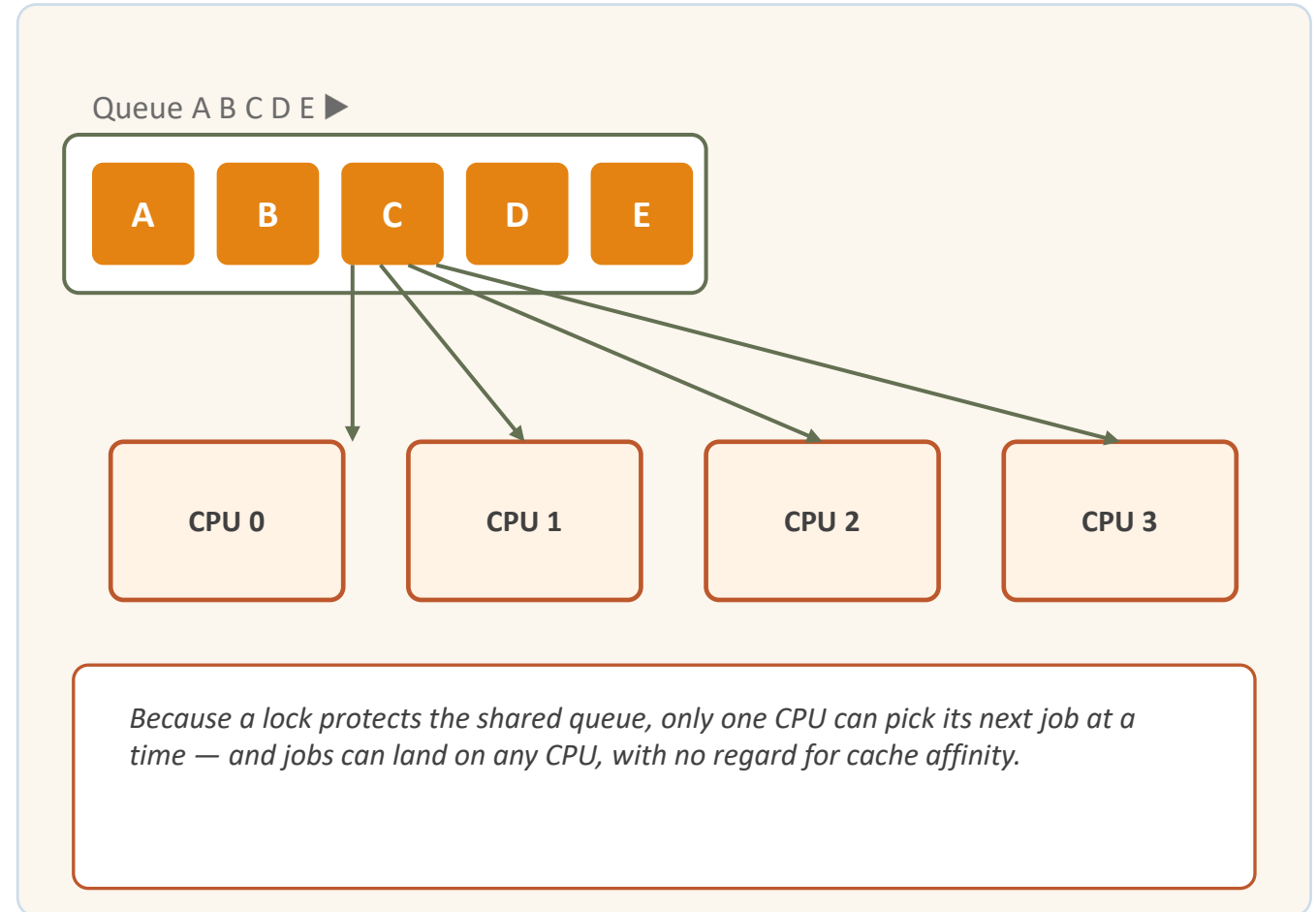
Different CPU each time — cold cache



State reloaded every time → process runs slower

SQMS: One Global Queue

- Put all jobs that need to be scheduled into a single, shared queue
- Reuses an existing single-CPU policy almost unchanged — simple to build
- Each CPU simply pulls the next job(s) to run from the shared queue
- Sounds great... but has two serious shortcomings, coming up next



SQMS Challenge: Locks and Lost Affinity

1. Lack of Scalability

To access the single shared queue correctly, the scheduler needs locks.

As CPU count grows, contention for that one lock grows — the system spends more time on lock overhead and less time doing real work.

2. Poor Cache Affinity

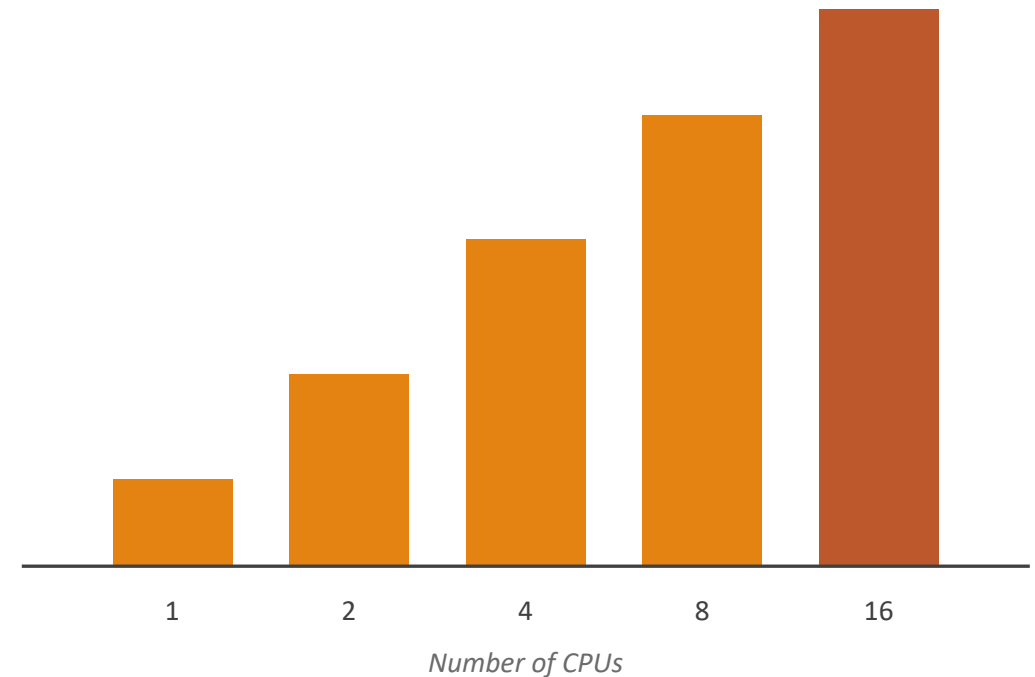
Every CPU pulls from the same queue — jobs bounce from CPU to CPU on every time slice, the opposite of what affinity wants:

CPU 0	A	E	D	C	B
CPU 1	B	A	E	D	C
CPU 2	C	B	A	E	D
CPU 3	D	C	B	A	E

(pattern repeats) — every job migrates on every tick

Lock contention grows with CPU count

(share of time spent on lock overhead, conceptual)



Conceptual trend — as CPUs scale, SQMS spends relatively less time doing useful work

SQMS: Adding Affinity Awareness

- Most SQMS schedulers add an affinity mechanism to reduce migrations
- Idea: keep most jobs on the same CPU; move only a few to balance load
- Below, only job E migrates each round — A, B, C, D each keep their CPU
- Next time, a different job can be chosen to migrate — “affinity fairness”
- **Trade-off: this scheme is straightforward to describe, but non-trivial to implement well**

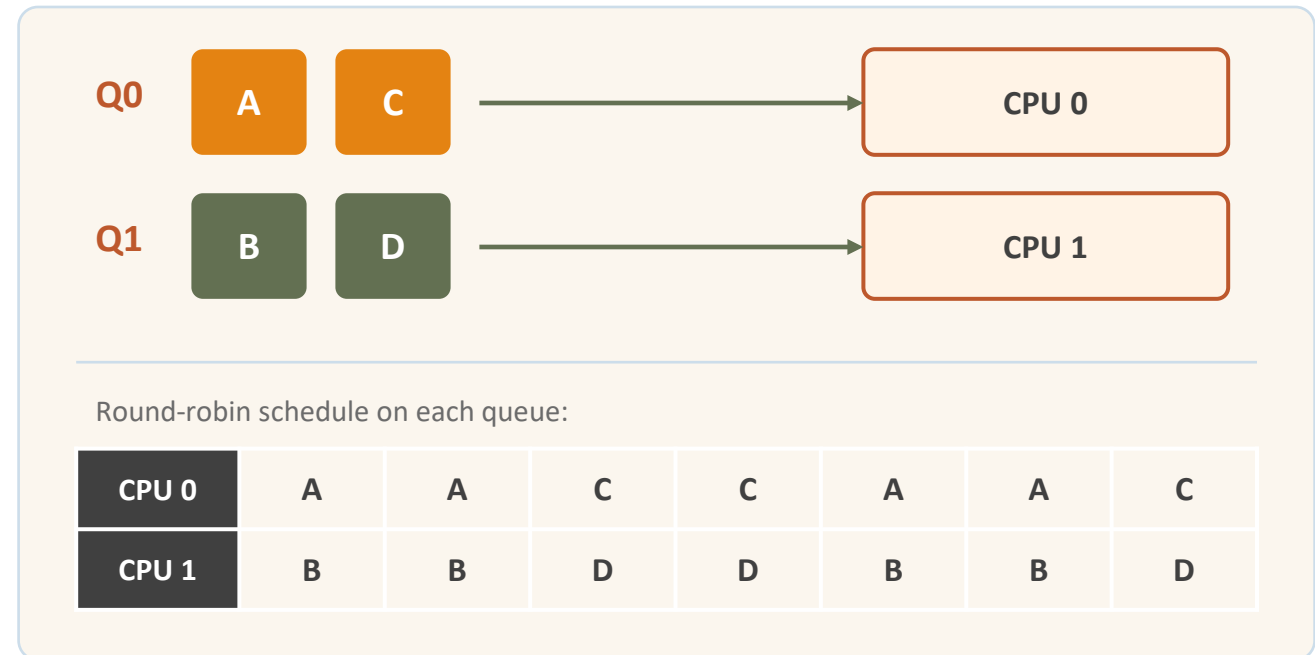
CPU 0	A	A	A	A	E
CPU 1	B	B	E	B	B
CPU 2	C	C	C	E	C
CPU 3	D	D	D	D	E

(pattern repeats) — job E (highlighted) is the only job that migrates across CPUs

A, B, C, D keep warm caches on their own CPU • only E pays the migration cost

MQMS: One Queue Per CPU

- Multiple scheduling queues — typically one per CPU
- Each queue follows its own discipline (e.g., round robin)
- A new job is placed on exactly one queue (random, or the shortest queue)
- Each CPU then schedules independently — no shared-queue lock, no cross-CPU sync



No shared lock to contend for, and jobs stay on their CPU — cache affinity comes for free.

But: what happens when one queue runs dry while another stays full? → See next slide.

MQMS: Why It Scales Better

1

Inherently More Scalable

As CPU count grows, so does the number of queues — lock and cache contention don't concentrate on one structure.

2

Cache Affinity “For Free”

Jobs stay on the CPU they were placed on, naturally reusing warm cache and TLB state.

3

Simple, Independent Scheduling

Each CPU runs its own familiar single-CPU policy (e.g., round robin) with no cross-CPU coordination needed.

CRUX

How should a multi-queue multiprocessor scheduler handle load imbalance, so as to still achieve its scheduling goals?

MQMS Challenge: Load Imbalance

Case 1 — job C finishes: $Q0 = \{ A \}$, $Q1 = \{ B, D \}$

CPU 0	A	A	A	A	A	A
CPU 1	B	B	D	D	B	B

A gets 2× the CPU time that B and D get — unfair!

Case 2 — A and C both finish: $Q0 = \{ \}$, $Q1 = \{ B, D \}$

CPU 0	—	—	—	—	—	—
CPU 1	B	B	D	D	B	B

CPU 0 sits completely idle while CPU 1 is fully loaded!

Root cause: each queue is scheduled independently, with no global view of load across CPUs.

- The obvious fix is migration — moving a job from an overloaded CPU's queue to an idle one
- A single migration is enough to fix Case 2 (just move B or D to CPU 0)
- But some situations (e.g., A alone vs. B and D alternating) need continuous migration to keep load balanced over time

Fixing Load Imbalance: Migration

Continuous migration example — A alone on CPU 0; B and D take turns visiting both CPUs to share load:

CPU 0	A	A	A	A	B	A	B	A	B	B	B	B
CPU 1	B	D	B	D	D	D	D	D	A	D	A	D

Jobs migrate between CPUs over time so that everyone gets a fair share.

Work Stealing

- A queue that is low on jobs (the source) occasionally peeks at another queue (the target)
- If the target is notably fuller, the source steals one or more jobs from it
- **Peek too often → high overhead, back to the scalability problem SQMS had**
- **Peek too rarely → risk of severe, lasting load imbalance**
- Finding the right peeking threshold is, in the words of OSTEP, “a black art”

Linux Multiprocessor Schedulers

No single solution has won out in the Linux community — three different schedulers have been used over the years.

Scheduler	Queue Design	Scheduling Approach	Primary Focus
O(1)	Multi-queue	Priority-based (like MLFQ)	Interactivity
CFS	Multi-queue	Deterministic proportional-share (like Stride)	Fairness
BFS	Single-queue	Proportional-share via EEVDF	Simplicity across cores

- Both O(1) and CFS use multiple queues — BFS is the outlier, using a single queue
- This shows both SQMS and MQMS approaches can be made to work in a real, widely deployed OS

SQMS vs. MQMS at a Glance

	Single-Queue (SQMS)	Multi-Queue (MQMS)
Simplicity	Simple — reuses single-CPU scheduler	Needs a job-placement policy per queue
Scalability	Poor — one lock, growing contention	Good — no shared, global lock
Cache Affinity	Poor by default; needs explicit fixes	Good by default — jobs stay put
Load Balance	Naturally balanced (one shared pool)	Can go idle / imbalanced without migration
Key Fix Needed	Affinity mechanisms	Migration / work stealing

Neither approach dominates — real schedulers ($O(1)$, CFS, BFS) show both can be engineered to work well.

Key Takeaways

Cache Coherence

Per-CPU caches can disagree on a value's contents; hardware (e.g., bus snooping) restores a single consistent view.

Locks Still Matter

Coherence hardware does not replace locks — concurrent updates to shared structures still need synchronization.

Cache Affinity

Re-running a job on the CPU it last used avoids reloading cache/TLB state, and is worth preserving.

SQMS Trade-off

Simple and load-balanced by nature, but suffers lock contention and poor affinity at scale.

MQMS Trade-off

Scales well and preserves affinity, but can suffer serious load imbalance without migration.

No Free Lunch

Building a general-purpose multiprocessor scheduler always means balancing these competing goals.

Thank You!!

Questions?

